

OrderPulse — Step 1: Infrastructure Setup

Technical Instruction & Execution Blueprint for AI Developer Agents

OBJECTIVE

Establish the foundational infrastructure layer using Docker Compose (PostgreSQL, Redis, RabbitMQ, Kafka, Loki, Grafana) with health checks and volume persistence.

Step 1 Tasks

- **1. Project Directory Initialization:** Create `orderpulse/` root directory and `.gitignore`.
- **2. Docker Compose Configuration:** Define all required service containers with mapped ports, persistence volumes, and health check parameters.
- **3. Container Spin Up:** Run `docker compose up -d` and monitor container health statuses.
- **4. Connection Verification:** Validate network connections across PostgreSQL (5432), Redis (6379), RabbitMQ (15672/5672), Kafka (9092), and Grafana (3000).
- **5. Environment Setup:** Create `.env.example` with default parameters for Go microservices.

version: '3.8'

services:

postgres:

image: postgres:16-alpine

container_name: orderpulse-postgres

environment:

POSTGRES_USER: appuser

POSTGRES_PASSWORD: apppassword

POSTGRES_DB: orderpulse_db

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U appuser -d orderpulse_db"]

interval: 5s

timeout: 5s

retries: 5

redis:

image: redis:7-alpine

container_name: orderpulse-redis

ports:

- "6379:6379"

volumes:

- redis_data:/data

healthcheck:

test: ["CMD", "redis-cli", "ping"]

interval: 5s

timeout: 5s

retries: 5

rabbitmq:

image: rabbitmq:3-management-alpine

container_name: orderpulse-rabbitmq

ports:

- "5672:5672"

- "15672:15672"

environment:

RABBITMQ_DEFAULT_USER: guest

RABBITMQ_DEFAULT_PASS: guest

healthcheck:

test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]

interval: 5s

timeout: 5s

retries: 5

kafka:

image: apache/kafka:3.7.0

container_name: orderpulse-kafka

ports:

- "9092:9092"

environment:

```
KAFKA_NODE_ID: 1
KAFKA_PROCESS_ROLES: 'broker,controller'
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: 'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT'
KAFKA_LISTENERS: 'PLAINTEXT://:29092,CONTROLLER://:9093,PLAINTEXT_HOST://localhost:9092'
KAFKA_INTER_BROKER_LISTENER_NAME: 'PLAINTEXT'
KAFKA_ADVERTISED_LISTENERS: 'PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092'
KAFKA_CONTROLLER_LISTENER_NAMES: 'CONTROLLER'
KAFKA_CONTROLLER_QUORUM_VOTERS: '1@kafka:9093'
KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
KAFKA_LOG_DIRS: '/tmp/kraft-combined-logs'
KAFKA_CLUSTER_ID: 'MkU30EVbNTcwNTJENDM2Qk'
```

loki:

```
image: grafana/loki:2.9.4
container_name: orderpulse-loki
ports:
  - "3100:3100"
command: -config.file=/etc/loki/local-config.yaml
```

grafana:

```
image: grafana/grafana:10.4.0
container_name: orderpulse-grafana
ports:
  - "3000:3000"
environment:
  - GF_SECURITY_ADMIN_PASSWORD=admin
depends_on:
  - loki
```

volumes:

```
postgres_data:
redis_data:
```

Environment Variables Template (`.env.example`)

```
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_USER=appuser
POSTGRES_PASSWORD=apppassword
POSTGRES_DB=orderpulse_db
POSTGRES_DSN="postgres://appuser:apppassword@localhost:5432/orderpulse_db?sslmode=disable"

REDIS_ADDR=localhost:6379
REDIS_PASSWORD=

RABBITMQ_URL="amqp://guest:guest@localhost:5672/"
KAFKA_BROKERS=localhost:9092
LOKI_URL="http://localhost:3100/loki/api/v1/push"
```



Verification Commands

```
# Start container cluster
docker compose up -d

# Verify state
docker compose ps

# Check individual container readiness
docker exec -it orderpulse-postgres pg_isready -U appuser -d orderpulse_db
docker exec -it orderpulse-redis redis-cli ping
docker exec -it orderpulse-rabbitmq rabbitmq-diagnostics ping
curl -s http://localhost:3100/ready
```